

Stockage, routage et choix techniques

Réponses aux questions de l'encadrante – Système RAG pour hcp.ma

Structure d'accueil : Haut-Commissariat au Plan (HCP), Direction des Systèmes
d'Information Statistiques

Binôme : Ayman El Baida et Saad Mesbah

Période du stage : 1er juillet – fin août 2026

Note rédigée suite à la réunion du 17 juillet 2026 – 20 juillet 2026

Table des matières

1	Introduction	2
2	Stockage des données après collecte	2
2.1	État actuel et limites	2
2.2	Redis comme couche de cache – proposition	2
3	Pourquoi diviser la base de données	3
4	Le Routeur : distinguer chiffré et narratif, et gérer le cas mixte	3
4.1	Limite du modèle actuel	3
4.2	Proposition révisée : deux signaux indépendants	3
5	Benchmarking des choix techniques (ADR 0002)	5
5.1	Scraping : requests + BeautifulSoup	5
5.2	Extraction PDF : pdfplumber	5
5.3	Embeddings : BGE-M3	5
5.4	Index vectoriel : ChromaDB	6
5.5	Recherche lexicale : rank-bm25	6
5.6	Base des indicateurs : SQLite	6
5.7	Interface de démonstration : Streamlit	7
6	Synthèse et prochaines étapes	7

1 Introduction

Lors de la réunion du 17 juillet 2026, quatre questions ont été soulevées sur l'architecture de stockage et de routage du système : (1) comment et où sont stockées les deux natures de données produites par le pipeline d'ingestion (texte narratif et indicateurs chiffrés), et quelles en sont les limites ; (2) l'intérêt d'introduire Redis ; (3) pourquoi diviser la base de données en deux systèmes plutôt qu'un seul, le Routeur devant interroger chacun indépendamment ; (4) comment le Routeur distingue une question chiffrée d'une question narrative, et comment il gère le cas où une question relève des deux à la fois. Ce document répond aux quatre points, puis présente le benchmarking demandé sur les choix techniques initiaux de l'ADR 0002 (`docs/adr/0002-stack-technique-prototype.md`).

2 Stockage des données après collecte

2.1 État actuel et limites

Le pipeline produit deux natures de données distinctes (ADR 0001, `docs/adr/0001-separation-texte-indicateurs.md`), stockées dans deux systèmes différents :

Donnée	Stockage actuel	Limite connue
Indicateurs chiffrés	Table <code>indicateur</code> (SQLite, <code>db/schema.sql</code>)	Fichier unique, un seul processus écrivain à la fois (mode WAL permet des lectures concurrentes, pas des écritures concurrentes). Suffisant pour un prototype à faible concurrence ; deviendrait un goulot d'étranglement avec de nombreux utilisateurs simultanés en production.
Chunks de texte narratif	Prévu : ChromaDB (index vectoriel) + index BM25 en parallèle (ADR 0002)	ChromaDB local a un plafond pratique autour de quelques millions de vecteurs et filtre les métadonnées <i>après</i> la recherche par similarité (moins précis à grande échelle qu'un filtrage préalable). Très adapté au développement local et à un prototype, pas conçu pour plusieurs serveurs.
Fichiers bruts (PDF/XLSX/DOCX)	Disque, <code>data/raw/</code> , hors base de données et hors dépôt git (<code>.gitignore</code>)	Volume qui croît sans limite ni sauvegarde automatique ; regenerable en relançant le Scraper, donc pas critique au stade actuel, mais à surveiller si le corpus grossit significativement.

Le principe qui gouverne ce choix : chaque nature de donnée va dans le système optimisé pour son mode d'accès. Les indicateurs se cherchent par égalité exacte (« donne-moi la ligne où `code_bds='I3181'` et `periode='2025T4'` ») – le cas d'usage historique d'une base relationnelle. Les chunks se cherchent par proximité sémantique (« trouve les passages qui parlent du même sujet que cette question ») – ce qu'une base relationnelle ne fait pas nativement, et ce pour quoi un index vectoriel existe.

2.2 Redis comme couche de cache – proposition

L'ADR 0004 (`docs/adr/0004-api-bds-pour-les-indicateurs.md`, section « précisions du 15 juillet ») décrit déjà, sans le nommer, un schéma *cache-aside* : pré-remplissage nocturne des indicateurs curés, appel direct à l'API BDS si absent du cache local, puis écriture du résultat pour enrichir ce cache. Redis est l'outil standard pour implémenter exactement ce schéma : une base clé-valeur en mémoire, donc des lectures nettement plus rapides qu'une base sur disque comme SQLite, avec une expiration automatique par clé (TTL) – pertinent ici puisque les indicateurs BDS se mettent à jour

à un rythme connu (mensuel pour l'IPC et l'IPPI, par exemple), ce qui permet de fixer une durée de vie de cache alignée sur cette fréquence plutôt que de coder cette logique de péremption à la main.

Redis ne remplacerait pas SQLite mais viendrait *devant* : SQLite reste la source de vérité durable, celle qui porte la traçabilité (chaque indicateur relié à un `id_document`, exigence NF3 du cadrage) – un rôle que Redis ne remplit pas nativement, étant volatile par défaut. Redis jouerait le rôle d'une couche rapide pour les indicateurs consultés fréquemment, avec SQLite comme filet de sécurité derrière pour toute donnée absente du cache mémoire. C'est une évolution proposée, pas encore implémentée ; elle fera l'objet d'un ADR dédié (ADR 0006) si elle est retenue.

3 Pourquoi diviser la base de données

Deux arguments distincts justifient cette séparation, l'un fonctionnel, l'autre technique.

Argument fonctionnel (ADR 0001). Un institut statistique officiel ne peut pas se permettre qu'une recherche par similarité renvoie un chiffre approximatif à la place du chiffre officiel exact. Les indicateurs doivent donc passer par une requête exacte, jamais par une recherche floue – ce qui exclut d'office de les stocker uniquement dans un index vectoriel.

Argument technique (adéquation outil/usage). Même sans la contrainte d'exactitude, ce sont deux problèmes de recherche différents, chacun avec son outil de prédilection. Une base relationnelle excelle à répondre à une requête exacte et indexée ; elle n'est pas conçue pour comparer des vecteurs de similarité. Un index vectoriel excelle à trouver des contenus sémantiquement proches ; il n'est pas conçu pour des requêtes structurées exactes. Utiliser deux systèmes, chacun pour ce qu'il fait le mieux, est un principe connu sous le nom de *polyglot persistence*. Le coût : deux systèmes à maintenir plutôt qu'un seul. Le bénéfice : chacun fait bien son travail plutôt qu'un seul qui ferait les deux moyennement bien.

4 Le Routeur : distinguer chiffré et narratif, et gérer le cas mixte

4.1 Limite du modèle actuel

`src/routeur.py` est aujourd'hui un squelette : une énumération `TypeQuestion` à deux valeurs mutuellement exclusives, `CHIFFRE` ou `NOTION`, et une méthode `classifier(question)` non encore implémentée (prévue semaine 3, classification par un prompt LLM court). Ce modèle à choix unique ne gère pas correctement une question qui relève des deux à la fois – exactement le cas soulevé en réunion. Une question comme « pourquoi le chômage a-t-il augmenté ces derniers trimestres ? » a une composante chiffrée (le taux, par période) et une composante narrative (les causes, présentes dans les rapports). Forcer un choix unique conduit à mal traiter ce genre de question, quel que soit le choix fait.

4.2 Proposition révisée : deux signaux indépendants

La révision proposée remplace le choix unique par deux signaux booléens indépendants – « cette question a-t-elle une composante chiffrée ? » et « a-t-elle une composante narrative ? » – les deux pouvant être vrais simultanément.

Étape	Fonctionnement proposé
Signal chiffré	Comparer la question aux 832 libellés du catalogue BDS déjà disponible (<code>data/bds_catalogue.json</code>), par correspondance de mots-clés ou de similarité. Signal peu coûteux à calculer, réutilise une donnée déjà en place.
Signal narratif	Détection d'une intention explicative – mots déclencheurs (« pourquoi », « comment », « expliquer », « tendance ») ou, en V2, un prompt LLM court dédié.

Étape	Fonctionnement proposé
Dispatch	Si signal chiffré seul : LookupStructure uniquement. Si signal narratif seul : RetrievalReranker uniquement. Si les deux : les deux modules sont interrogés en parallèle.
Fusion	Générateur compose la réponse à partir de ce qui a été trouvé – le chiffre exact d’un côté, l’explication sourcée de l’autre – avec citation des deux sources si les deux chemins ont produit un résultat.

Cette révision n’invalide pas le travail déjà fait : **LookupStructure** et **RetrievalReranker** restent chacun responsables d’un seul type de recherche (ADR 0001 inchangé). Ce qui change, c’est uniquement la couche de décision au-dessus – le Routeur peut désormais activer les deux chemins pour une même question plutôt que de choisir l’un ou l’autre. Cette révision sera documentée formellement dès le début de son implémentation (semaine 3, Sprint 3).

5 Benchmarking des choix techniques (ADR 0002)

Comparatif demandé des choix initiaux face aux alternatives courantes, sur la base de comparatifs indépendants récents et, quand elle existe, de l'expérience réelle déjà accumulée sur ce projet.

5.1 Scraping : requests + BeautifulSoup

Option	Évaluation
requests BeautifulSoup (retenu)	+ Léger, rapide à mettre en œuvre, suffisant tant que les pages sont du HTML statique généré côté serveur – confirmé empiriquement : les 27 pages testées le 15 juillet se sont récupérées et parsées sans exécution JavaScript nécessaire.
Scrapy	Framework complet (concurrence, files de reprise, pipelines de traitement intégrés). Justifié pour des milliers de pages avec crawl récurrent programmé ; surdimensionné pour les quelques dizaines de pages ciblées par ce projet.
Playwright Selenium	/ Nécessaire uniquement si le contenu est généré côté client par JavaScript (site « dynamique»). Non justifié ici : vérifié en conditions réelles que hcp.ma sert du HTML directement exploitable.

5.2 Extraction PDF : pdfplumber

Option	Évaluation
pdfplumber (retenu)	Contrôle fin de la détection de tableaux, adapté aux mises en page ambiguës – précisément le cas des rapports hcp.ma. Déjà validé en conditions réelles : 30 PDF authentiques traités sans erreur le 17 juillet 2026, de 2 à 645 tableaux extraits selon le fichier. Cette preuve empirique pèse plus que n'importe quel comparatif théorique.
PyMuPDF (fitz)	Le plus rapide pour l'extraction de texte brut à grande échelle ; détection de tableaux plus récente et moins fine que pdfplumber sur des mises en page complexes. Utilisé en <i>repli</i> dans l'ADR 0002 si pdfplumber montre ses limites de performance (les 2 fichiers volumineux « Tabulations » identifiés comme lents).
Camelot	Meilleure précision de détection sur des tableaux très structurés (factures, rapports à grille nette), mais dépendance supplémentaire (Ghostscript) et moins robuste sur des mises en page variées comme celles observées sur hcp.ma.

5.3 Embeddings : BGE-M3

Option	Évaluation
BGE-M3 (retenu)	Multilingue (plus de 100 langues, dont français et arabe – pertinent si le support de l'arabe est un jour activé), open source (licence MIT, hébergeable localement, aucun coût par appel ni dépendance à une API externe). Combine recherche dense, parcimonieuse et multi-vecteurs (ColBERT) dans un seul modèle, avec un contexte de 8 000 tokens. Sur les classements MTEB récents, légèrement en retrait par rapport à OpenAI text-embedding-3-large sur les tâches uniquement anglaises, mais nettement au-dessus sur les tâches multilingues.

Option	Évaluation
OpenAI text-embedding-3-large	Très bon score MTEB, infrastructure gérée sans installation. Mais : API payante par appel, dépendance externe (pertinent au regard du point encore ouvert de l'ADR 0002 sur la politique de sécurité des données du HCP), et les documents restent envoyés à un tiers externe.
multilingual-e5-large	Alternative open source multilingue sérieuse, performances proches de BGE-M3 sur plusieurs tâches, mais sans le support natif dense+parcimonieux+multi-vecteurs qui simplifie l'implémentation de la recherche hybride prévue.

5.4 Index vectoriel : ChromaDB

Option	Évaluation
ChromaDB (retenu)	Local, installation quasi instantanée, idéal pour le développement et un prototype – plafond pratique autour de quelques millions de vecteurs, très au-dessus du volume attendu pour ce projet (corpus de quelques dizaines à quelques centaines de documents).
FAISS	Le plus rapide en recherche brute, mais c'est une bibliothèque de calcul, pas une base de données : pas de persistance ni de gestion des métadonnées intégrée, à construire soi-même par-dessus.
Qdrant	Conçu pour la production : filtrage des métadonnées appliqué <i>avant</i> la recherche (plus correct à grande échelle), scalable horizontalement. Pertinent si HCP déploie ce système en production avec un corpus en forte croissance – chemin de migration naturel depuis ChromaDB si ce cap est atteint.
pgvector	Extension PostgreSQL : pas de nouveau système si une base PostgreSQL est déjà en place. À envisager si l'infrastructure HCP standardise sur PostgreSQL plutôt que sur des bases spécialisées.

5.5 Recherche lexicale : rank-bm25

Option	Évaluation
rank-bm25 (retenu)	Bibliothèque Python pure (dépendance : NumPy uniquement), aucun serveur à installer ou maintenir. Adapté aux corpus de moins de 100 000 documents – très au-dessus du volume de ce projet.
BM25S	Évolution récente de rank-bm25 , même simplicité d'usage mais nettement plus rapide (calcul matriciel creux). À considérer comme mise à niveau directe si la vitesse de recherche devenait un jour un point sensible – aucun changement d'architecture requis.
Elasticsearch / OpenSearch	Standard en production à grande échelle (millions de documents, plusieurs nœuds), mais nécessite un serveur Java dédié et une charge opérationnelle largement hors de proportion avec le corpus actuel.

5.6 Base des indicateurs : SQLite

Option	Évaluation
SQLite (retenu)	Conforme ACID, mode WAL permettant des lectures concurrentes, zéro maintenance (pas de serveur à déployer), très rapide pour des recherches ponctuelles par clé exacte – exactement le scénario de LookupStructure . Limite : un seul écrivain à la fois, adapté à une faible concurrence.
PostgreSQL	Supporte de nombreux écrivains simultanés, choix par défaut pour une application multi-utilisateurs en production. À envisager pour un déploiement réel à HCP au-delà du prototype.
DuckDB	Optimisé pour l’analytique sur de gros volumes (requêtes d’agrégation, colonnes plutôt que lignes) – ne correspond pas au scénario ici, qui est de la recherche ponctuelle de lignes individuelles, pas de l’analyse en masse.

5.7 Interface de démonstration : Streamlit

Option	Évaluation
Streamlit (retenu)	Plus flexible pour une interface multi-écrans (question/réponse, mais aussi visualisation d’indicateurs éventuelle) et une personnalisation poussée de la mise en page.
Gradio	Chemin le plus rapide pour un simple chatbot – composants prêts à l’emploi pour l’interaction conversationnelle. Pertinent si le besoin se limite strictement à un fil de discussion, sans tableau de bord annexe.

6 Synthèse et prochaines étapes

Les quatre points soulevés en réunion aboutissent à trois actions concrètes : (1) documenter formellement l’introduction éventuelle de Redis comme couche de cache devant SQLite, sous forme d’un ADR 0006 ; (2) réviser `routeur.py` pour remplacer le choix unique **CHIFFRE/NOTION** par deux signaux indépendants capables de se combiner, avant le début de son implémentation prévue en semaine 3 ; (3) le choix des briques techniques actuelles (ADR 0002) reste justifié au stade prototype pour chacune d’elles, avec pour chacune un chemin de montée en charge identifié si HCP décidait d’un déploiement en production au-delà de ce stage.